Justin K

April 9, 2026

Contents

0.1 Initial Setup . . . 1
0.2 Entra Setup . . . 1
0.3 Securing the ASP.NET Core API in Entra . . . 2
0.4 Dependency Injection And Database Contexts . . . 2
0.5 Add a shared project in Blazor . . . 2
0.5.1 Create the Razor Class Library (RCL) . . . 3
0.5.2 Add Project References . . . 3
0.5.3 Register the Shared Namespace . . . 3
0.5.4 Special Considerations . . . 3

0.1 Initial Setup

1. dotnet new list
   Lists templates for starting new applications.
2. You can view files in the project by using either the *File Explorer* or the *Solution Explorer*.
3. *Program.cs* creates an instance of the *app* object, which is an instance of the *WebApplication* object that is the hosting application.
4. *app* starts an HTTP server for your application so that you can start listening to HTTP requests.
5. The *builder* object can be used to configure services for the application.
6. The *Build()* method introduces the *Kestrel Web Server*, reading *appsettings.json*.
7. `<Project Sdk="Microsoft.NET.Sdk.Web">`
   The above from the *.csproj* file imports all the libraries required for a web API application.
8. dotnet build
   to build the application from your solution directory.
9. Use a *record* for a *Dto*, since it represents a contract between a client and a server as to how data will be transferred and this data should be immutable

0.2 Entra Setup

1. Log into Entra ID.
2. Select the tenant for the application. To begin with, set the organizational directory for the tenant as single-tenant.
3. The tenant should have at least one user with the role "Application Developer".
4. The backend API will accept tokens that have been generate by only our tenant.
5. Every API that you register in Entra is going to need a *scope* that defines the permissions required by a client that requires access to this API.

6. Scopes are defined by the *Expose an API* section in Entra. every scope is prefixed with *api://*.

0.3 Securing the ASP.NET Core API in Entra

1. Each endpoint that is restricted should have *.RequireAuthorization()* appended.
2. The ASP.NET API back-end code needs to be able to validate the access tokens that will be attached to API requests that it receives.
   This requires NuGet package *Microsoft.AspNetCore.Authentication.JwtBearer*.
3. An *Authentication Scheme* defines which handler to use to validate incoming access tokens.
4. Update the middleware in the ASP.NET Core application for authentication and authorization.
5. To authorize the backend API to make requests, an access token will need to be attached to each request. To get this application token, a client application has to be registered with Entra.
6. The client application can be Postman, for testing purposes. The Redirect URI is `https://oauth.pstmn.io/v1/callback`.
   Register Postman as an App in Entra.
7. Entra has to be notified that Postman can talk to the ASP.NET Core back-end API.
8. Postman has to be granted API permissions in Entra to access the scope defined in the back-end API.

0.4 Dependency Injection And Database Contexts

1. DbContext is not thread-safe. Setting the lifetime of DbContext to *Scoped* ensures that the connections do not span across HTTP requests.
2. DbContexts are an expensive resource. A scoped lifetime for DbContexts helps ensure that resources are used wisely.
3. *Scoped* lifetime services are created once per HTTP request and reused within that HTTP request.
4. An environment variable can be set in PowerShell to hide the connection string during production, so that this string is not coded in the *appsettings.json* file.

   ```
   $env:ConnectionStrings__DefaultConnection="DefaultConnection": "Data Source=
       localhost,1433;Persist Security Info=True;User ID=sa;Password=Mssql2025!;
       Pooling=False;MultipleActiveResultSets=False;Encrypt=True;
       TrustServerCertificate=True;Application Name=\"SQL Server Management Studio
       \";Command Timeout=0;Database=RESTAPI;"
   ```

   The code in *Program.cs* does not change. Setting the environment variable as shown above will add the connection string *"ConnectionStrings:DefaultConnection"* to *builder.Configuration*.

0.5 Add a shared project in Blazor

To create a shared project in Blazor, the standard and recommended approach is to use a Razor Class Library (RCL). This allows you to share components, pages, styles, and logic across different Blazor hosting models (Server, WebAssembly, or Hybrid).

0.5.1 Create the Razor Class Library (RCL)

Visual Studio: Right-click your solution, select Add ¿ New Project, search for "Razor Class Library," and name it (e.g., MySharedComponents).
Note: Ensure you do not check "Support pages and views" unless you specifically need Razor Pages/MVC support, as it is unnecessary for Blazor-only projects. **.NET CLI**: Run the following command in your solution folder: dotnet new razorclasslib -o MySharedComponents.

0.5.2 Add Project References

To use the shared code, your main Blazor application must reference the RCL:
In Solution Explorer, right-click the Dependencies node of your main Blazor project.
Select Add Project Reference....
Check the box for your newly created RCL (e.g., MySharedComponents) and click OK.

0.5.3 Register the Shared Namespace

To make components from the shared library available throughout your application:
Open the `_Imports.razor` file in your main Blazor project.
Add a @using statement for the shared library's namespace:
@using MySharedComponents.

0.5.4 Special Considerations

1. **Static Assets (CSS/JS)**: Files in the RCL's wwwroot folder are accessible via the path `_content/{LIBRARY_NAME}/{FILE_PATH}`. For example:
   `<link href="_content/MySharedComponents/styles.css" rel="stylesheet" />`.

2. **Routable Pages**: If you include pages with @page directives in the RCL, you must tell the Blazor Router where to find them by adding the library's assembly to the AdditionalAssemblies parameter in App.razor or Routes.razor.

3. **Legacy "Shared" Project:** Older ".NET Core Hosted" WebAssembly templates automatically created a Shared project for data models. In modern Blazor Web Apps (.NET 8+), use a standard Class Library or RCL to achieve the same result.